CPSC 121
Lab 07 – Shipping Boxes

Lab Description

Overview

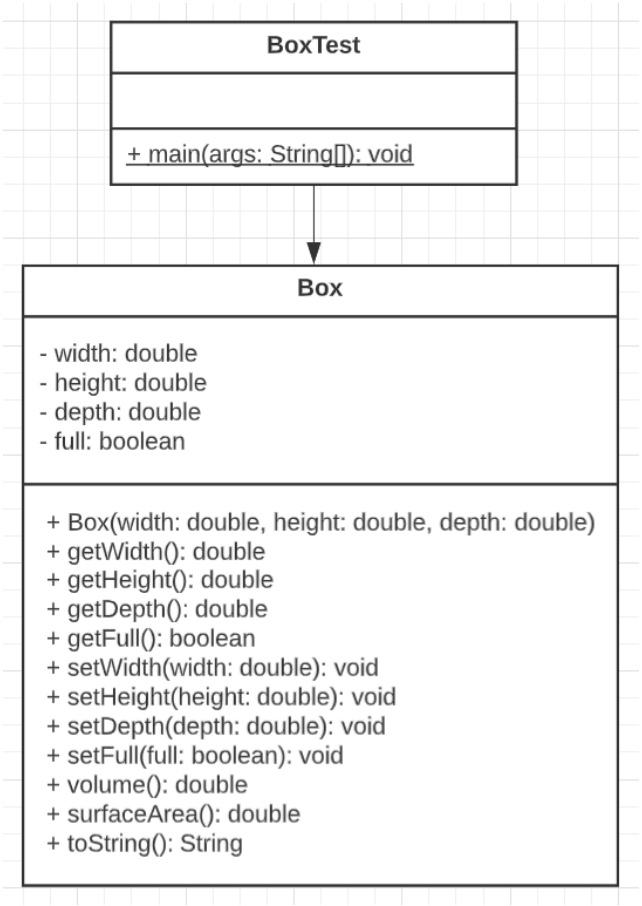
To implement and test your own class using:

- Properties (i.e. state),
- Encapsulation (visibility),
- A constructor,
- Getter methods,
- Setter methods,
- Public class methods, and
- An appropriate `toString` method.

Getting Started

1. Create a lab project
2. Create a new class called `Box` and a new class called `BoxTest`

UML Diagram



Part 1: Write your own `Box` Class

Design and implement a class called `Box` that represents a 3-dimensional box.

1. Instance Data
   Your Box class will need properties to store the size of the box. All of these properties will be of type '`private double`'.
   - `width`
   - `height`
   - `depth`

   Your `Box` class will also need a property to keep track of whether or not the box is full. This will be of type '`private boolean`'.
   - `full`
2. Constructor
   Declaration: `public Box( double width, double height, double depth)`
   - Your `Box` constructor will accept the `width, height,` and `depth` of the box.
   - In your constructor, initialize the `width, height,` and `depth` properties to values passed in as parameters to the constructor.
   - Each newly created box will be empty. This means that you will also need to initialize `full` to false in your constructor.
3. Getter and Setter Methods
   Include getter and setter methods for all instance data. We will assume that all supplied dimension values are positive values. Here is an example of the method declarations for width to get you started.
   - Getter: `public double getWidth()`
   - Setter: `public void setWidth(double width)`
4. Public Methods
   - Write a `public double volume()` method that will calculate and return the volume of the box based on the instance data values.
   - Write a `public double surfaceArea()` method that will calculate and return the surface area of the box based on the instance data values.
5. `toString` Method
   Write a `public String toString()` method that returns a one-line description of the box. The description should provide its dimensions and whether or not the box is full. Format all double values to 2 decimal places.

   Example: `An empty 4.00 x 5.00 x 2.00 box.`

Part 2: Testing your Class

Now, create a driver class called `BoxTest` and do the following in the main method. Follow these directions **EXACTLY**.

Create your first box

1. Create a `Box` variable called `smallBox` and instantiate it with width 4, height 5, and depth 2.
2. Print the one-line description of `smallBox` using its `toString()` method.
3. Confirm that `smallBox`'s width, height, and depth getter methods are returning the correct values.
4. Confirm that `smallBox's volume()` and `surfaceArea()` methods are returning the correct values.
5. Use `smallBox`'s setter methods to change it from "empty" to "full" and to change its dimension values to width 2, height 3, and depth 1.
6. Print the one-line description of `smallBox` using `toString()` again, and confirm that all changed values are reflected.
7. Confirm that all getter and other methods return the correct values reflecting `smallBox`'s current dimensions and full value.

Create several boxes and find the largest one

Do not delete what you did above! Add the following code to the end of your `BoxTest` class.

1. Declare and instantiate an `ArrayList` object that will store your `Box` objects. See the class sidebar slides on the **ArrayList and For-Each** for examples of how to use the `ArrayList`. **NOTE: You will be doing this with `Box` objects.**

2. In a **standard** "for" loop that iterates 5 times, create a new `Box` with randomly generated width, height, and depth and add the box to your `ArrayList`. Limit dimension values to a reasonable range. Randomly generate a Boolean value and use it to call the `Box`'s method for setting "full" (use the `nextBoolean()` method in the `Random` class). **Please note: this step is NOT involving the for-each loop described above!**

3. In the loop, print out the `Box`'s details, labeled according to the loop iteration (i.e. if this is the first iteration of the loop, label the box as "Box 1: " in the output).

4. Use a for-each loop (as shown in the listing above) to find the `Box` with the largest volume. You will need to declare a largest box variable of type `Box` to keep track of the largest box. This variable needs to be declared outside of the loop so that it will still be available when the loop ends. Use conditional statements inside the loop to compare the current Box to the largest Box, so far, and update the largest box variable when appropriate. **NOTE: this step IS using the for-each loop described above!**

5. After the loop, print the details of the largest `Box`. Confirm that you program identified the correct box.

Expected Output:

```
An empty 5.00 x 4.00 x 2.00 box
smallbox's width = 5.0
smallbox's height = 4.0
smallbox's depth = 2.0
smallbox's volume = 40.0
smallbox's surface area = 76.0
smallbox reports its full status as: false

========Change smallbox using it's setters========

A full 2.00 x 3.00 x 1.00 box
smallbox's width = 2.0
smallbox's height = 3.0
smallbox's depth = 1.0
smallbox's volume = 6.0
smallbox's surface area = 22.0
smallbox reports its full status as: true

========Create 5 boxes========

Box 0: An empty 53.61 x 36.95 x 18.26 box
Box 1: An empty 81.35 x 85.54 x 89.90 box
Box 2: An empty 86.27 x 71.20 x 97.98 box
Box 3: A full 6.98 x 95.04 x 5.28 box
Box 4: An empty 12.43 x 16.87 x 55.47 box

========Find the largest box========

Largest Box
An empty 81.35 x 85.54 x 89.90 box with volume 625516.44 and surface area 43920.96
```

Rubric

- Do the programs work as described? (Needs my sign-off): **10 points**
- Did you follow the directions along with style and coding practices: **10 points**
- **Total: 20 points**

Submitting the Lab

Compress the submission files (see below) into a .zip file named:
Lab07_*LastName_FirstName*.zip When you are done and ready to submit, submit the following files in the .zip file:

- `Box.java`
- `BoxTest.java`